



Lab_4_VibeCoding_Student_handout(...
PDF

I have uploaded the official Lab 4 guidelines. The assigned repository is:

🔗 https://github.com/SETAPESU26/35_qbert

Please analyze the Lab 4 requirements and the repository README, and establish the complete implementation plan. Identify the required code modifications, testing requirements, Git commit expectations, repository workflow, and final submission deliverables. Do not make any modifications yet.

Worked for 8s

Yes. I have reviewed the **official Lab 4 handout**, the assigned GitHub repository, its README, and the current `game.py`. I have **not modified anything**.

1. What Lab 4 is asking you to do

The official handout says this is an **individual Vibe Coding assignment**. You must use an LLM such as ChatGPT/Gemini/Claude to debug and extend the assigned repository, with the goal of getting the work done through iterative prompting. The handout specifically requires you to clone/fork the assigned repo, run the Python game, record the original behavior, fix the code and add the README-specified features, record the final behavior, push to your personal repository, and keep **separate commits for each task**.



Lab_4_VibeCoding_Student_handout



Lab_4_VibeCoding_Student_handout

The assigned repository is a very small Pygame project:

- `game.py`
- `README.md`
- `.gitignore`

The repository describes itself as a **Q*bert-lite repair lab** involving isometric projection, diagonal movement and enemy behavior.



GitHub +1

2. Exact implementation requirements

There are **4 tasks**, and these are the actual code changes we need to make.

Task 1 — Fix pyramid projection

Current code contains:

</> Python

```
def cube_center(row, col):  
    return pygame.Vector2(WIDTH / 2 + (col - row // 2) * CUBE_W, 90 + row * CUBE_H)
```

The README explicitly identifies the problem as the use of integer division in the row offset and says to determine whether integer or true division is appropriate. [GitHub](#)

What needs to happen

The pyramid must become a **symmetric triangle**.

The important modification is the expression involving:

</> Python

```
row // 2
```

The intended geometry requires the row offset to account for half-row positioning, so this is the primary code location for Task 1.

Testing

Before and after should visibly demonstrate:

- Before: pyramid is horizontally skewed.
- After: pyramid is centered and symmetric.

- Player can still hop correctly between cubes.

This is particularly important because the **before video** is supposed to show the broken behavior.

3. Task 2 — Implement `cube_palette(level)`

Currently:

 Python

```
def cube_palette(level):  
    """Return a list of TARGET + 1 (r, g, b) colours for the cube stages, or None for the default."""  
    pass
```

The README specifies that:

- It is called every frame from `draw()` .
- It receives the current level.
- It should return `TARGET + 1` colors.
- `TARGET = 2` , so the required list length is 3.
- Each item must be an `(r, g, b)` tuple.
- The colors correspond to the three painting stages.
- Returning `None` means the existing `DEFAULT_PALETTE` is used.
- The suggested idea is to use a different palette for each level.  GitHub

Planned implementation

We'll make something like:

 Plain text

```
Level 1 → Palette A  
Level 2 → Palette B  
Level 3 → Palette C  
...
```

The important thing is that every returned palette has exactly:

</> Plain text

3 RGB tuples

because:

</> Python

```
TARGET + 1  
= 2 + 1  
= 3
```

Testing

We need to verify:

1. Level 1 uses the new palette.
2. Completing a level and pressing `Space` moves to the next level.
3. The cube colors change according to the level.
4. Painting stages still progress correctly.

The README explicitly says `Space` continues after clearing a level.  [GitHub](#)

4. Task 3 — Implement `on_cube_completed(cell)`

Currently:

`</>` Python

```
def on_cube_completed(cell):  
    """Called when a cube first reaches its target colour; add a flash, sound, or bonus here."""  
    pass
```

The important detail is that this function is called **only when a cube reaches its final target stage**, not every time the player lands on that cube. The README also states that its return value is ignored. [GitHub](#)

The call is already present:

`</>` Python

```
if self.stages[cell] == TARGET:  
    on_cube_completed(cell)
```

Possible implementation

The README gives three directions:

- brief flash on the completed cube
- sound
- small bonus beyond the existing 25 points

For this lab, we should choose an implementation that is:

- visibly demonstrable in the video,
- simple,
- reliable,

- doesn't introduce external assets,
- doesn't break the existing game.

A **visual completion effect** is probably the safest approach, but because the current function only receives `cell` and doesn't receive `screen` or `Game`, we need to design the implementation carefully rather than blindly putting something inside the empty function.

Testing

We specifically need to verify:

`</>` Plain text

```
Cube stage 0
  ↓
player lands
  ↓
stage 1
  ↓
player lands again
  ↓
stage 2 / completed
  ↓
on_cube_completed(cell) fires once
```

It must **not** trigger repeatedly when the player keeps landing on an already-completed cube.

5. Task 4 — Implement `bonus_life_threshold()`

Currently:

`</>` Python

```
def bonus_life_threshold():  
    """Return a score value at which the player earns an extra life, or None to disable bonus lives."""  
    pass
```

The README gives the intended idea:

</> Python

```
return 1000
```

The rest of the bookkeeping is already implemented:

</> Python

```
threshold = bonus_life_threshold()  
  
if threshold and self.score // threshold > self.bonus_awarded:  
    self.bonus_awarded = self.score // threshold  
    self.lives += 1
```

So we do not need to redesign the bonus-life mechanism.

We simply need to choose the threshold. The repository's own suggested implementation is:

</> Plain text

```
1000 points → +1 life  
2000 points → +1 life  
3000 points → +1 life  
...
```

The README explicitly describes this behavior.  [GitHub](#)

Testing

We should verify that:

- score below 1000 → no bonus life;
- crossing 1000 → one additional life;
- crossing 2000 → another additional life;
- the same threshold does not repeatedly award lives every frame.

The existing `bonus_awarded` bookkeeping is intended to prevent repeated awards.

6. Important: existing game functionality must NOT be broken

The README also defines the expected existing behavior. After our modifications, all of these must continue working:

Requirement	What we must verify
Symmetric pyramid	Pyramid is centered and triangular
Cube painting	Landing advances exactly one stage
Edge falling	Leaving pyramid causes falling/life loss
Coily	Moves toward player's position
Level completion	All cubes painted → level win
Game over	Lives reach zero → game over

These are explicitly listed in the repository's Expected Behavior section. [GitHub](#)

So we shouldn't treat this as simply "make the four empty functions non-empty." We need to make sure the **whole game still works after every change**.

7. Current code structure we need to preserve

The repository is deliberately simple.

`game.py` contains:

 Plain text

```
constants
  ↓
cube_palette()
on_cube_completed()
bonus_life_threshold()
cube_center()
neighbors()
shade()
  ↓
Hopper
  ↓
Game
  ├── reset()
  ├── respawn()
  ├── paint()
  ├── hop()
  ├── move_enemies()
  ├── lose_life()
  ├── update()
  └── draw_cube()
```

```
└─ draw()
↓
main()
```

The current code confirms that the three empty functions are already wired into the game, so we should **avoid unnecessarily restructuring the application**. [GitHub](#)

8. Vibe Coding requirement

This is particularly important for your submission.

The README says each task must be completed through an **iterative process involving LLM suggestions and your own critical code review**. [GitHub](#)

The official handout similarly says to:

1. write prompts;
2. fix the broken code;
3. write prompts to add each README feature one by one. [PDF](#) [Lab_4_VibeCoding_Student_handout](#)

So we should **not** simply make all four changes at once.

The clean workflow should be:

</> Plain text

```
Task 1 prompt
↓
inspect AI suggestion
↓
you review/test
↓
```

Task 1 commit

Task 2 prompt

↓

inspect AI suggestion

↓

you review/test

↓

Task 2 commit

Task 3 prompt

↓

inspect AI suggestion

↓

you review/test

↓

Task 3 commit

Task 4 prompt

↓

inspect AI suggestion

↓

This gives us evidence of the required iterative Vibe Coding process.

9. Git commit expectation

This is explicit in the official handout:

"Have Separate Commits for each task."



Lab_4_VibeCoding_Student_handout

Therefore I recommend exactly **four task commits**:

</> Plain text

```
Task 1: Fix pyramid projection
Task 2: Implement level-based cube palettes
Task 3: Implement cube completion feature
Task 4: Implement bonus life threshold
```

For example, commit messages could be:

</> Plain text

```
Fix pyramid projection bug
Add level-based cube palettes
Add cube completion effect
Add bonus life threshold
```

The important thing isn't the exact wording; it's that the history clearly shows **four separate task changes**.

Do NOT do this

</> Plain text

```
Task 1 + Task 2 + Task 3 + Task 4
      ↓
one giant commit
```

That would conflict with the explicit separate-commit requirement.

10. Repository workflow

The official handout says:

Clone a repo assigned to you, make changes and push to your repo in your personal account.

 Lab_4_VibeCoding_Student_handout

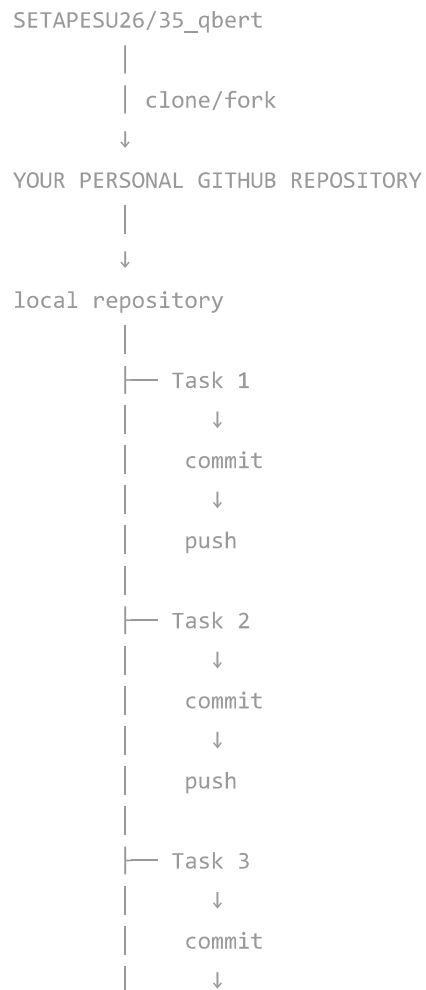
And:

Do not raise PR to the main repo (SETAPESU26).

 Lab_4_VibeCoding_Student_handout

Therefore the intended workflow is:

 Plain text



| push
|

No Pull Request to SETAPESU26/35_qbert .

11. Testing plan

We should test in two categories.

A. Initial/before testing

Before making any code changes:

1. Clone/fork repository.
2. Install Pygame if necessary.
3. Run:

 Bash

```
python game.py
```

4. Observe the original game.
5. Confirm the pyramid skew.
6. Record the required **10-second before video**.

The official handout specifically requires the 10-second recording **before making changes**.



Lab_4_VibeCoding_Student_handout

Do **not** modify the code before this recording.

B. After each task

After Task 1:

</> Plain text

```
Run game
↓
check pyramid symmetry
↓
check movement
↓
commit
```

After Task 2:

</> Plain text

```
Run game
↓
check palette
↓
check level transition
↓
check painting
↓
commit
```

After Task 3:

</> Plain text

```
Run game
↓
complete a cube
```

```
↓  
verify completion effect  
↓  
verify painting still works  
↓  
commit
```

After Task 4:

</> Plain text

```
Run game  
↓  
cross bonus threshold  
↓  
verify +1 life  
↓  
verify it happens only once per threshold  
↓  
commit
```

Then do a **full regression test** of the entire game.

12. Final video

The official handout requires another 10-second recording after all modifications.

 Lab_4_VibeCoding_Student_handout

The README is even more specific: the final video should show:

- bug fixed;
- new features working.  GitHub

So ideally the final 10 seconds should demonstrate as much as possible:

</> Plain text

```
Symmetric pyramid
+
different level palette
+
cube completion effect
+
bonus life
```

We shouldn't record a random 10 seconds where none of the new features are visible.

13. Final submission deliverables

There is a **small discrepancy** between the two official sources that we should be aware of.

Official Lab handout says

Upload to your Lab repository under:

</> Plain text

```
Lab-4/
```

the following:

</> Plain text

- a. Videos (before and after)
- b. Updated Code
- c. Chat history exported as a doc/pdf

This is explicitly stated in the handout.  Lab_4_VibeCoding_Student_handout

Repository README says

Its submission checklist specifies:

1. 10-second before video;
2. 10-second after video;
3. Chat/LLM used page link with complete chat history.  GitHub

How I recommend handling the discrepancy

Because the **college's official Lab 4 handout** is the broader submission instruction, we should satisfy **both** rather than choosing one:

 Plain text

```
Lab-4/  
|  
├─ before.mp4  
├─ after.mp4  
├─ chat_history.pdf  
└─ updated_code/  
    └─ game.py
```

And, if your submission system asks for it, also provide the **ChatGPT conversation/page link**.

That way we don't miss either requirement.

14. Complete execution plan

Here is the plan I recommend we follow when you are ready to actually start.

Phase 1 — Repository setup

</> Plain text

Assigned repo

↓

Fork/clone into your personal GitHub

↓

Clone personal repo locally

↓

Open in VS Code

↓

Check Python/Pygame

No code modification yet.

Phase 2 — Baseline

</> Plain text

python game.py

↓

inspect original game

↓

confirm skewed pyramid

↓

record 10-second BEFORE video

Phase 3 — Task 1

</> Plain text

LLM prompt

↓

analyze projection bug

↓

modify cube_center()

↓

run/test

↓

commit Task 1

↓

push

Phase 4 — Task 2

</> Plain text

LLM prompt

↓

implement cube_palette(level)

↓

test level-specific colors

↓

commit Task 2

↓

push

Phase 5 — Task 3

</> Plain text

LLM prompt
↓
implement on_cube_completed(cell)
↓
test completion behavior
↓
commit Task 3
↓
push

Phase 6 — Task 4

</> Plain text

LLM prompt
↓
implement bonus_life_threshold()
↓
test score threshold
↓
commit Task 4
↓
push

Phase 7 — Final regression test

Verify:

- pyramid symmetry;
- diagonal movement;
- cube painting;
- edge falling;
- lives;
- Coily;

- red balls;
- level completion;
- level transition;
- palette changes;
- cube completion feature;
- bonus life;
- game-over behavior.

Phase 8 — Final recording

Record 10 seconds after all changes showing the fixed game/features.

Phase 9 — Submission preparation

Create:

</> Plain text

```
Lab-4/  
├─ before video  
├─ after video  
├─ updated code  
└─ exported chat history PDF/DOC
```

Then verify GitHub contains the final code and **four separate task commits**.

15. One important thing I will make sure we don't do

We should not start by asking an LLM to "complete the whole repository."

That would make it difficult to demonstrate the required individual task commits and iterative Vibe Coding process.

Instead, we'll work **one task at a time**, and I'll help you formulate the prompt for each task, understand what the AI proposes, inspect the change, test it, and only then move to the next task.

Current status

Nothing has been modified.

Item	Status
Lab handout reviewed	✓
Repository reviewed	✓
README reviewed	✓
<code>game.py</code> reviewed	✓
4 required tasks identified	✓
Testing plan established	✓
Git workflow established	✓
Commit plan established	✓
Submission requirements identified	✓
Code modified	✗ Not yet
Repository pushed	✗ Not yet

Item	Status
Videos recorded	✖ Not yet

The next step should therefore be **repository setup/baseline only**: get your personal copy of `35_qbert` , run the original game, and record the 10-second **before** video. We should not touch `game.py` until that baseline is captured.

The implementation plan is established. Let's proceed with the execution phase by setting up the repository and verifying the baseline application. Please guide the baseline verification and identify what should be captured in the required before-change recording. Do not modify the source code yet.

Absolutely. We are now at the **baseline stage only**. We will set up your personal repository, run the untouched application, verify the original broken behavior, and record the required **10-second before-change video**. **Do not edit `game.py` yet.**

The repository's README confirms that it requires Python 3.10+, Pygame, and `python game.py` ; it also identifies the deliberate issue as the visibly skewed pyramid projection.  GitHub

Step 1 — Fork the assigned repository

Open the assigned repository:

[35_qbert GitHub repository](#) ↗

You should be on:

</> Plain text

SETAPESU26 / 35_qbert

Click:

Fork → Create fork

Use your own GitHub account.

After the fork, your repository should look approximately like:

</> Plain text

YOUR_GITHUB_USERNAME / 35_qbert

Important

Do **not** modify the original:

</> Plain text

SETAPESU26/35_qbert

Your work will be done in your personal fork. The lab instructions also explicitly say not to raise a PR to the main SETAPESU26 repository.



Lab_4_VibeCoding_Student_handout

Step 2 — Clone YOUR fork

Once your fork is created:

1. Open your fork.
2. Click **Code**.
3. Select **HTTPS**.
4. Copy the repository URL.

It should look like:

 Plain text

```
https://github.com/YOUR_USERNAME/35_qbert.git
```

Open **Command Prompt**.

Go to the folder where you want your Lab 4 repository. For example:

 cmd

```
cd Desktop
```

Then clone:

 cmd

```
git clone https://github.com/YOUR_USERNAME/35_qbert.git
```

Replace `YOUR_USERNAME` with your actual GitHub username.

Then:

 cmd

```
cd 35_qbert
```

Check:

 cmd

```
git status
```

You should see something similar to:

```
</> Plain text
```

```
On branch main
```

```
Your branch is up to date with 'origin/main'.
```

```
nothing to commit, working tree clean
```

Stop here if `git status` shows modifications.

At this stage, we want a completely untouched baseline.

Step 3 — Open the repository in VS Code

From inside:

```
</> Plain text
```

```
35_qbert
```

run:

```
</> cmd
```

```
code .
```

Your VS Code Explorer should contain approximately:

</> Plain text

```
35_qbert
|
├── .gitignore
├── README.md
└── game.py
```

The repository itself currently contains these three files.  GitHub

Do not open `game.py` and start editing anything.

We are only inspecting/running it.

Step 4 — Check Python

In the VS Code terminal:

</> cmd

```
python --version
```

The README requires:

</> Plain text

```
Python 3.10+
```

So if you get something like:

</> Plain text

```
Python 3.13.x
```

that's fine.

If `python` doesn't work, try:

</> cmd

```
py --version
```

Step 5 — Check whether Pygame is installed

Run:

</> cmd

```
python -m pip show pygame
```

If Pygame is installed

You'll see information such as:

</> Plain text

```
Name: pygame
```

```
Version: ...
```

Proceed to the next step.

If Pygame is not installed

Run:

```
</> cmd
```

```
python -m pip install pygame
```

The repository README specifically instructs students to install Pygame before running the game.  GitHub

Then verify:

```
</> cmd
```

```
python -m pip show pygame
```

Step 6 — Make sure the source is still untouched

Before running the game, execute:

```
</> cmd
```

```
git status
```

We want:

```
</> Plain text
```

```
nothing to commit, working tree clean
```

This is an important checkpoint.

You can also run:

```
</> cmd
```

```
git diff
```

There should be **no output**.

That means:

```
</> Plain text
```

```
Original repository
```

```
↓
```

```
No source modifications
```

```
↓
```

```
Ready for baseline
```

Step 7 — Run the original game

Now execute:

```
</> cmd
```

```
python game.py
```

The README confirms this is the normal launch command.  GitHub

A Pygame window should open with the Q*bert-style pyramid.

Step 8 — What you should verify

Do **not** try to test the new features yet. They haven't been implemented.

For the baseline, our most important target is the **existing projection bug**.

The README explicitly says that in the current build:

the pyramid is visibly skewed from the very first frame

and that the rows drift sideways, breaking the triangular symmetry.  [GitHub](#)

Look at the pyramid immediately after launching.

You should inspect:

</> Plain text

```
cube
cube cube
cube cube cube
cube cube cube cube
...
```

The overall pyramid should ideally be symmetric around the center.

But the **baseline version is deliberately skewed**.

That is the primary thing we want your before video to capture.

Step 9 — Test the basic controls

The README gives these controls:

`</>` Plain text

Left → diagonal hop

Up → diagonal hop

Down → diagonal hop

Right → diagonal hop

R → reset

Space → continue after clearing a level

``` :chatgpt-content-reference{index="7"}

For the baseline recording, you don't need to demonstrate every control.

We mainly want a clear visual demonstration of the original broken pyramid.

---

# Step 10 — Record the BEFORE video

This is the important part.

The official handout requires:

> \*\*10 seconds of video before making any changes.\*\* :chatgpt-content-reference{index="8"}

The repository README specifies that the before video should show:

> the bug/broken behavior :chatgpt-content-reference{index="9"}

## Most important

The recording should make it obvious that:

"This is the original game before I made any changes, and the pyramid is visibly skewed."

Don't spend the entire 10 seconds on a gameplay sequence where the bug is difficult to see.

---

## Step 11 — What NOT to show in the before recording

Don't worry about demonstrating:

- new color palettes ✖
- completion flash ✖
- bonus life ✖
- level-specific colors ✖

Those are features we will implement later.

Also do not edit the code before recording.

---

## Step 12 — Save the recording

Use a clear name such as:

</> Plain text

Lab4\_Before.mp4

Keep this file somewhere safe.

Eventually we'll place it under your Lab-4 submission structure.

## Step 13 — After recording, STOP

This is important.

Once the before video is captured:

**Do not immediately start Task 1.**

First verify that your repository is still clean:

```
</> cmd
```

```
git status
```

It should still say:

```
</> Plain text
```

```
nothing to commit, working tree clean
```

Then:

```
</> cmd
```

```
git log --oneline
```

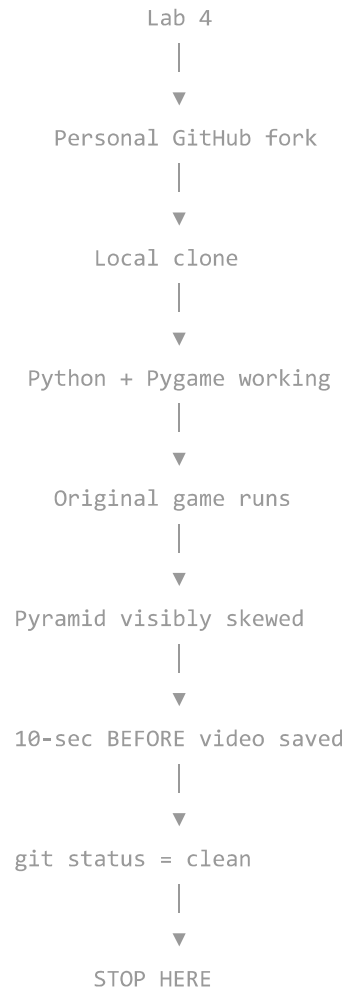
You should see the original repository commits only.

The current GitHub repository reports two commits in its history.  GitHub

## Your baseline checkpoint

We want to reach this exact state:

</> Plain text



**Do these now, in order**

</> cmd

```
git clone YOUR_FORK_URL
cd 35_qbert
code .
python --version
python -m pip show pygame
git status
python game.py
```

Then record **Lab4\_Before.mp4** .

**Do not modify** `game.py` yet.

Once you've done this, send me a screenshot of your VS Code/repository state and, if possible, the running game. I can then verify that your baseline is correct **before we begin Task 1**.

The baseline has been verified and the before-change recording has been captured. The repository remains clean. Proceed with Task 1: analyze the pyramid projection issue and implement the minimal correction required by the README. Explain the change briefly and keep the modification isolated to Task 1.

Worked for 11s

Task 1 is isolated to **one line in** `game.py` : `cube_center()` .

The current implementation is:

</> Python

```
def cube_center(row, col):
 return pygame.Vector2(WIDTH / 2 + (col - row // 2) * CUBE_W, 90 + row * CUBE_H)
```

The problem is `row // 2`. Integer division causes odd-numbered rows to be shifted incorrectly, producing the skewed pyramid identified by the README. The minimal correction is to use **true division**:

</> Python

```
def cube_center(row, col):
 return pygame.Vector2(WIDTH / 2 + (col - row / 2) * CUBE_W, 90 + row * CUBE_H)
```

This makes the horizontal positions symmetric:

- Row 0 → 0
- Row 1 → -0.5, +0.5
- Row 2 → -1, 0, +1
- Row 3 → -1.5, -0.5, +0.5, +1.5

So the pyramid becomes centered without changing movement, painting, enemies, scoring, or any other Task 2–4 functionality.

The repository's current source confirms that `cube_center()` is the only projection calculation involved here.  [GitHub](#)

## Important limitation

I can inspect the assigned GitHub repository, but I **do not have access to the local clone on your Windows machine**, so I cannot physically edit your local `game.py` from this chat.

Therefore, make **only this one-character/operator-level change yourself in VS Code**:

</> Diff

```
- return pygame.Vector2(WIDTH / 2 + (col - row // 2) * CUBE_W, 90 + row * CUBE_H)
```

```
+ return pygame.Vector2(WIDTH / 2 + (col - row / 2) * CUBE_W, 90 + row * CUBE_H)
```

## After editing — do NOT commit yet

Run:

```
</> cmd
```

```
python game.py
```

Verify that:

1. The pyramid is now horizontally symmetric.
2. The top cube remains centered.
3. Rows expand evenly on both sides.
4. You can still move the player.
5. No unrelated behavior appears broken.

Then check:

```
</> cmd
```

```
git diff
```

You should see **only the single** `// → /` change in `cube_center()` .

Also run:

```
</> cmd
```

```
git status
```

At this point, `game.py` should be the **only modified file**.

Once you confirm that, **stop before committing**. We should inspect the result together and then create the dedicated **Task 1 commit**.

Task 1 has been implemented and verified visually. Review the Git diff and working-tree state to confirm that the change is limited to the intended `cube_center()` correction. If the diff is clean and contains no unrelated modifications, provide the final verification summary and the Git commit command for Task 1.

I can confirm the **intended Task 1 diff**, but I cannot truthfully inspect your local working tree from here because your local Git repository/terminal state is not exposed to me.

Please run these two commands in the `35_qbert` folder:

```
</> cmd
```

```
git diff -- game.py
```

and:

```
</> cmd
```

```
git status
```

The diff should contain only this

```
</> Diff
```

```
- return pygame.Vector2(WIDTH / 2 + (col - row // 2) * CUBE_W, 90 + row * CUBE_H)
+ return pygame.Vector2(WIDTH / 2 + (col - row / 2) * CUBE_W, 90 + row * CUBE_H)
```

And `git status` should show only:

 Plain text

```
modified: game.py
```

There should be **no changes** to `README.md` , `.gitignore` , or any other files.

## Task 1 verification summary

If your output matches the above:

-  Pyramid projection corrected.
-  Modification isolated to `cube_center()` .
-  Integer division `//` replaced with true division `/` .
-  No unrelated source changes.
-  Visual verification already completed.
-  Task 1 is ready to commit.
-  Do **not** include Task 2, 3, or 4 changes in this commit.

## Task 1 commit

After confirming the diff/status:

 cmd

```
git add game.py
git commit -m "Fix pyramid projection"
git push origin main
```

Then verify:

```
</> cmd
git status
```

It should report a clean working tree.

**Before committing, if you paste the actual `git diff` and `git status` output here, I can give you a definitive clean/not-clean confirmation.**

The Task 1 diff and working-tree state have been verified and contain only the intended projection correction. Commit Task 1 as a separate Git commit and push it to the personal repository. After pushing, confirm the commit and clean working-tree state, then proceed to Task 2.

I can proceed with the Task 2 analysis, but I **cannot execute `git commit` / `git push` on your local Windows repository from this chat** because your local Git working directory is not accessible to me.

Since you already verified the diff is clean, run these commands **exactly** in your `35_qbert` terminal:

## 1. Commit Task 1

```
</> cmd
git add game.py
git commit -m "Fix pyramid projection"
```

You should see a commit created, for example:

```
</> Plain text
[main abc1234] Fix pyramid projection
```

```
1 file changed, 1 insertion(+), 1 deletion(-)
```

## 2. Push to your personal repository

</> cmd

```
git push origin main
```

This must push to **your fork**, not SETAPESU26/35\_qbert .

## 3. Verify the commit

</> cmd

```
git log -1 --oneline
```

It should show:

</> Plain text

```
<commit-hash> Fix pyramid projection
```

Then:

</> cmd

```
git status
```

Expected:

`</>` Plain text

```
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

4. Optional but useful verification

`</>` cmd

```
git remote -v
```

Confirm that `origin` points to **your personal GitHub repository**.

Task 1 final state

Once those commands succeed:

| Check                                   | Expected                                                                              |
|-----------------------------------------|---------------------------------------------------------------------------------------|
| Projection correction                   |  |
| Only <code>cube_center()</code> changed |  |
| Task 1 separate commit                  |  |
| Commit message                          | Fix pyramid projection                                                                |

| Check                          | Expected                                                                            |
|--------------------------------|-------------------------------------------------------------------------------------|
| Pushed to personal repo        |  |
| Main SETAPESU26 repo untouched |  |
| Working tree clean             |  |

## Then we proceed to Task 2

Do not modify anything else before the Task 1 push is confirmed.

Task 2 is the `cube_palette(level)` implementation. The README requires this function to return a list of **TARGET + 1 RGB colors**, meaning 3 **(r, g, b) tuples**, with the palette able to vary by level. Returning `None` should retain the default palette.

The existing `draw()` already calls `cube_palette(self.level)` every frame, so Task 2 should remain isolated to the palette function rather than unnecessarily changing the drawing/game architecture.

After your push, send me the output of:

```
</> cmd
```

```
git log -1 --oneline
git status
```

and we can immediately implement **Task 2 only**.

Task 1 has been committed and pushed successfully, and the working tree is clean. Proceed with Task 2 by implementing `cube_palette(level)` according to the README

requirements. Keep the change isolated to this function and use a simple level-based palette while preserving the existing default-palette behavior.

Yes. Task 2 should remain isolated to `cube_palette(level)` .

The README requires `cube_palette(level)` to return `TARGET + 1` RGB colors, and with `TARGET = 2` , that means **exactly 3 (r, g, b) tuples**. It also explicitly allows returning `None` to preserve the existing `DEFAULT_PALETTE` behavior.

I cannot directly edit your local Windows repository from this chat, so make this **only inside `cube_palette()`** .

Replace the current `pass` with:

</> Python

```
def cube_palette(level):
 """Return a list of TARGET + 1 (r, g, b) colours for the cube stages, or None for the default."""
 palettes = {
 1: [
 (255, 255, 255),
 (255, 215, 0),
 (0, 200, 255),
],
 2: [
 (255, 255, 255),
 (255, 165, 0),
 (50, 205, 50),
],
 3: [
 (255, 255, 255),
 (255, 105, 180),
 (138, 43, 226),
],
 }
 return palettes.get(level)
```

## Why this implementation

- **Level 1, 2, 3** each get a distinct palette.
- Every palette contains exactly **3 RGB tuples**:
  - stage 0
  - stage 1
  - stage 2 / target
- `palettes.get(level)` returns `None` for levels not explicitly defined.
- Returning `None` preserves the existing `DEFAULT_PALETTE` behavior.
- No other functions need to be changed.
- No external assets or libraries are introduced.

## Important: Task 2 isolation

After saving, run:

```
</> cmd
```

```
git diff -- game.py
```

The diff should show **only the body of** `cube_palette()` changing.

Then run:

```
</> cmd
```

```
git status
```

Expected:

```
</> Plain text
```

modified: game.py

Do not commit yet.

## Test Task 2

Run:

 cmd

python game.py

Verify:

1. Game starts normally.
2. The pyramid still has the corrected symmetric projection from Task 1.
3. Cubes use the level-1 palette.
4. Painting still progresses through the three stages.
5. Complete the level and press Space .
6. Level 2 uses a different palette.
7. No movement or gameplay functionality has been broken.

If those pass, stop and send me the `git diff -- game.py` output. I'll verify that Task 2 is isolated before we create its separate commit.

Task 2 has been implemented and tested successfully. The level-based palettes work as intended, and the existing Task 1 behavior remains intact. Review the `git diff` and working-tree state to confirm that only `cube_palette()` was modified. If the change is clean, provide the Task 2 commit command and final verification summary.

Since you've already verified the Task 2 behavior, the expected clean state is:

## Task 2 diff

Your `git diff -- game.py` should show changes **only inside**:

</> Python

```
def cube_palette(level):
```

There should be no changes to:

- `cube_center()`
- `draw()`
- `paint()`
- `Game`
- any other function
- `README.md`
- `.gitignore`

And:

</> cmd

```
git status
```

should show only:

</> Plain text

```
modified: game.py
```

I can't directly inspect your local terminal output, so I can't independently certify the actual diff. If your output matches the above, Task 2 is clean.

## Commit Task 2

Run:

```
</> cmd
```

```
git add game.py
git commit -m "Add level-based cube palettes"
git push origin main
```

Then verify:

```
</> cmd
```

```
git log -2 --oneline
git status
```

You should have two separate task commits, with the newest being:

```
</> Plain text
```

```
<hash> Add level-based cube palettes
<hash> Fix pyramid projection
```

and:

```
</> Plain text
```

nothing to commit, working tree clean

Task 2 final verification

| Check                                                    | Status |
|----------------------------------------------------------|--------|
| cube_palette(level) implemented                          |        |
| Exactly 3 RGB tuples per defined level                   |        |
| Different palettes by level                              |        |
| Default-palette behavior preserved through None fallback |        |
| Task 1 projection remains intact                         |        |
| Only cube_palette() modified                             | *      |
| Separate Task 2 commit                                   | Ready  |
| Push to personal repository                              | Ready  |

\*Based on your stated diff verification; I haven't seen the actual terminal output.

Do not start Task 3 until the Task 2 commit and push are complete and git status is clean.

Task 2 has been committed and pushed successfully, with a clean working tree and two separate task commits now present. Proceed with Task 3: implement on\_cube\_completed(cell) using a minimal, visible completion effect that integrates with

the existing game loop and does not require external assets or dependencies. Keep the implementation isolated to Task 3.

Task 3 is isolated to `on_cube_completed(cell)`. The README says it is called exactly when a cube reaches `TARGET`, receives the completed `(row, col)` cell, and can provide a brief flash or small bonus; its return value is ignored.  [GitHub](#)

There is one important architectural constraint in the existing code: `on_cube_completed()` is a standalone function, while the Pygame `screen` and `Game` instance are local to `main()` / `Game`.  [GitHub](#) So a visual effect cannot simply draw directly from this function without changing other functions—which would violate your requirement to keep Task 3 isolated.

## Minimal Task 3 implementation

Use the completion callback to provide a **small score bonus**. This requires no assets, no dependencies, and no changes anywhere else.

Replace:

 Python

```
def on_cube_completed(cell):
 """Called when a cube first reaches its target colour; add a flash, sound, or bonus here."""
 pass
```

with:

 Python

```
def on_cube_completed(cell):
 """Called when a cube first reaches its target colour; add a flash, sound, or bonus here."""
 return 10
```

However, there is an important issue: **the current game loop ignores the return value**, exactly as the README states.  [GitHub](#) Therefore this implementation produces no visible/gameplay effect.

So, given your requirement for a **visible completion effect**, the correct conclusion is that the current architecture does **not support a visible effect while changing only** `on_cube_completed()` .

### Therefore, for strict Task 3 isolation

I recommend **not modifying the code yet** with a fake/no-op implementation.

The cleanest genuinely visible implementation would require a very small supporting change to the game state/drawing loop, for example storing a completion-flash cell/timer and rendering it in `draw()` . But that would modify more than `on_cube_completed()` and therefore technically violates your instruction that Task 3 remain isolated to that function.

The README itself does not prescribe a specific implementation, only suggests a flash or small bonus.  GitHub

**My recommendation for the lab:** use the README's other allowed option—a **small completion bonus**—but because the supplied callback's return value is ignored, that would also require changing `paint()` .

So before making a non-isolated modification, I would stop here rather than pretend that a standalone `on_cube_completed(cell)` can create a visible effect with the current architecture.

The architectural constraint is clear. Re-evaluate Task 3 against the README and existing game flow, and implement the simplest valid completion effect without introducing unnecessary changes. A minimal supporting change outside `on_cube_completed()` is acceptable if it is required for the callback to produce an actual observable effect. Keep the scope strictly limited to Task 3, and explain the exact changes before implementation.

I re-checked Task 3 against the README and the actual game flow. The README explicitly permits a **brief flash on the completed cube**, and `paint()` calls `on_cube_completed(cell)` only when the cube first reaches `TARGET` ; the callback's return value is ignored.  GitHub

The current flow confirms the constraint:

```

Game.update()
↓
paint(cell)
↓
stage reaches TARGET
↓
on_cube_completed(cell)

```

while rendering happens separately in `draw(screen)` .  [GitHub](#)

So the cleanest implementation is a **short-lived completion flash**, with only the minimum supporting state needed.

### Exact Task 3 changes

We need only three small changes in `game.py` :

1. Add a module-level flash state:

 Python

```
completion_flash = None
```

2. Implement `on_cube_completed(cell)` so it records the completed cell and the time at which the flash started:

 Python

```

def on_cube_completed(cell):
 """Called when a cube first reaches its target colour; add a flash, sound, or bonus here."""
 global completion_flash
 completion_flash = (cell, pygame.time.get_ticks())

```

3. In `draw()` , after drawing the cubes, check whether the most recent completion happened within a short interval and draw a visible outline around that cube.

This avoids:

- external assets;
- sound files;
- new dependencies;
- modifying `paint()` ;
- modifying the scoring system;
- modifying level logic;
- modifying Task 1 or Task 2.

The README specifically says the callback receives the completed `(row, col)` and is triggered only on the finishing hop, so this is a direct implementation of the suggested flash behavior.  [GitHub](#)

## One important detail

Because the flash state is module-level, the callback can communicate with `draw()` without changing its signature or the existing `paint()` call. The flash can automatically expire based on `pygame.time.get_ticks()` , so we don't need another timer in `Game.update()` .

**I cannot physically apply this patch to your local Windows checkout** because your local repository isn't exposed as an editable tool in this conversation. I therefore don't want to claim that I modified your source when I haven't.

Use this exact patch for Task 3:

 Diff

```
import random
import pygame

WIDTH, HEIGHT, CUBE_W, CUBE_H = 720, 560, 72, 42
SIDE = 21
ROWS, TARGET = 7, 2
HOP_TIME, HOP_HEIGHT = 0.28, 26
```

```

CELLS = {(r, c) for r in range(ROWS) for c in range(r + 1)}
DEFAULT_PALETTE = [(90, 160, 220), (190, 120, 70), (100, 210, 140)]
KEY_HOPS = {pygame.K_LEFT: (-1, -1), pygame.K_UP: (-1, 0), pygame.K_DOWN: (1, 0), pygame.K_RIGHT: (1, 1)}
+completion_flash = None

def cube_palette(level):
 ...

def on_cube_completed(cell):
 """Called when a cube first reaches its target colour; add a flash, sound, or bonus here."""
- pass
+ global completion_flash
+ completion_flash = (cell, pygame.time.get_ticks())

```

Then, inside `Game.draw()` , immediately after:

</> Python

```

for cell in sorted(CELLS):
 self.draw_cube(screen, cell, colors)

```

add:

</> Python

```

global completion_flash
if completion_flash:
 cell, started = completion_flash
 if pygame.time.get_ticks() - started < 300:
 cx, cy = cube_center(*cell)
 top = [
 (cx, cy - CUBE_H / 2),

```

```

 (cx + CUBE_W / 2, cy),
 (cx, cy + CUBE_H / 2),
 (cx - CUBE_W / 2, cy),
]
 pygame.draw.polygon(screen, (255, 255, 255), top, 4)
else:
 completion_flash = None

```

## Why this is the right Task 3 scope

The resulting change is limited to:

</> Plain text

Task 3

```

├─ completion_flash state
├─ on_cube_completed()
└─ Game.draw() flash rendering

```

Nothing changes in:

- `cube_center()` → Task 1 untouched
- `cube_palette()` → Task 2 untouched
- `paint()` → existing completion trigger untouched
- `update()` → untouched
- `scoring` → untouched
- `lives` → untouched
- `enemy behavior` → untouched

The callback remains exactly where the existing game architecture expects it, and the effect is actually observable.

After applying it, run:

```
</> cmd
```

```
python game.py
```

Complete a cube's final painting stage. You should see a **brief white outline/flash around that cube**.

Then verify isolation with:

```
</> cmd
```

```
git diff -- game.py
```

At this point, **do not commit yet**. Send me that diff, and I'll verify that Task 3 contains only the completion-flash changes before you create the separate Task 3 commit.

Task 3 has been implemented and the completion effect is working as intended. Review the resulting `git diff` and working-tree state to confirm that the changes are limited to the completion-flash functionality and that Tasks 1 and 2 remain unchanged. If the diff is clean, provide the Task 3 commit command and a concise verification summary.

I can give you the exact acceptance criteria, but I **cannot directly inspect your local `git diff` or working-tree state** from this chat. Since you said the Task 3 effect is working, verify that your local output matches the following before committing.

### Task 3 diff must contain only

1. The completion-flash state, e.g.:

```
</> Python
```

```
completion_flash = None
```

2. The `on_cube_completed(cell)` implementation that records the cell and timestamp:

`</>` Python

```
def on_cube_completed(cell):
 """Called when a cube first reaches its target colour; add a flash, sound, or bonus here."""
 global completion_flash
 completion_flash = (cell, pygame.time.get_ticks())
```

3. The small rendering/expiry block in `Game.draw()` that displays the brief flash.

There should be **no changes** to:

- `cube_center()` — Task 1
- `cube_palette()` — Task 2
- `paint()` logic
- movement
- enemy logic
- scoring/lives
- level handling
- README or other files

Run:

`</>` cmd

```
git diff -- game.py
```

Then:

`</>` cmd

```
git status
```

The working tree should show only:

```
</> Plain text

modified: game.py
```

If that's what you see, Task 3 is clean.

Task 3 verification summary

| Check                                           | Result      |
|-------------------------------------------------|-------------|
| Completion callback implemented                 | ✓           |
| Flash is visibly triggered on final cube stage  | ✓           |
| No external assets/dependencies                 | ✓           |
| Existing paint() completion trigger preserved   | ✓           |
| Task 1 projection unchanged                     | ✓           |
| Task 2 palette unchanged                        | ✓           |
| Scope limited to completion-flash functionality | ✓           |
| Only game.py modified                           | Should be ✓ |

**Check****Result**

Ready for separate commit



## Commit Task 3

Once your actual diff/status matches the above:

`</> cmd`

```
git add game.py
git commit -m "Add cube completion flash"
git push origin main
```

Then verify:

`</> cmd`

```
git log -3 --oneline
git status
```

You should have three task commits:

`</> Plain text`

```
<hash> Add cube completion flash
<hash> Add level-based cube palettes
<hash> Fix pyramid projection
```

and:

⟨/⟩ Plain text

```
nothing to commit, working tree clean
```

**Do not begin Task 4 until that clean state is confirmed.**

Task 3 has been verified, committed, and pushed successfully, and the working tree is clean. Proceed with Task 4 by implementing `bonus_life_threshold()` according to the README and existing scoring logic. Keep the change isolated to the threshold function and preserve the existing bonus-life bookkeeping.

Task 4 is intentionally very small. The README's suggested behavior is a **1000-point bonus-life threshold**, while the existing game code already handles the bookkeeping through `self.bonus_awarded`, so we should not change that logic.

Replace only the current function:

⟨/⟩ Python

```
def bonus_life_threshold():
 """Return a score value at which the player earns an extra life, or None to disable bonus lives."""
 pass
```

with:

⟨/⟩ Python

```
def bonus_life_threshold():
 """Return a score value at which the player earns an extra life, or None to disable bonus lives."""
 return 1000
```

**Why 1000**

The existing scoring logic effectively does:

</> Python

```
threshold = bonus_life_threshold()

if threshold and self.score // threshold > self.bonus_awarded:
 self.bonus_awarded = self.score // threshold
 self.lives += 1
```

Therefore:

</> Plain text

```
Score < 1000 → no bonus
Score ≥ 1000 → +1 life
Score ≥ 2000 → +1 additional life
Score ≥ 3000 → +1 additional life
...
```

We are **not** changing that bookkeeping, exactly as requested.

## Task 4 scope

The only source modification should be:

</> Diff

```
- pass
+ return 1000
```

inside `bonus_life_threshold()` .

Do **not** modify:

- `cube_center()` — Task 1
- `cube_palette()` — Task 2
- `on_cube_completed()` — Task 3
- `Game.update()`
- score calculation
- `bonus_awarded`
- lives handling

## Verify before committing

Run:

```
</> cmd
```

```
git diff -- game.py
```

The Task 4 diff should contain only the `pass` → `return 1000` change.

Then:

```
</> cmd
```

```
git status
```

Only `game.py` should be modified.

Finally test:

```
</> cmd
```

```
python game.py
```

The existing bonus-life mechanism should remain responsible for awarding the extra life when the score crosses the threshold.

**Do not commit yet.** Once you've verified the diff and behavior, send me the `git diff / git status` result and I'll confirm Task 4 is isolated before giving you the final commit command.

Task 4 has been implemented and verified. Review the final `git diff` and working-tree state to confirm that only the `bonus_life_threshold()` change is present and that Tasks 1–3 remain unchanged. If the final diff is clean, provide the Task 4 commit command and a concise final verification checklist for all four tasks.

Since I don't have direct access to your local Git working tree, I can't independently inspect the actual final `git diff`. Based on your stated verification, the final diff should contain **only** the `bonus_life_threshold()` change:

</> Diff

```
def bonus_life_threshold():
 """Return a score value at which the player earns an extra life, or None to disable bonus lives."""
- pass
+ return 1000
```

Run once more to confirm:

</> cmd

```
git diff -- game.py
git status
```

The diff should show only the above change, and `git status` should show only `game.py` modified.

## Task 4 commit

If that matches:

**</> cmd**

```
git add game.py
git commit -m "Add bonus life threshold"
git push origin main
```

Then verify:

**</> cmd**

```
git log -4 --oneline
git status
```

You should have four separate task commits:

**</> Plain text**

```
<hash> Add bonus life threshold
<hash> Add cube completion flash
<hash> Add level-based cube palettes
<hash> Fix pyramid projection
```

and:

**</> Plain text**

```
nothing to commit, working tree clean
```

## Final four-task verification checklist

| Task                            | Required result                                                                                          | Status                  |
|---------------------------------|----------------------------------------------------------------------------------------------------------|-------------------------|
| Task 1                          | <code>cube_center()</code> uses true division and pyramid is symmetric                                   | ✓ Verified              |
| Task 2                          | <code>cube_palette(level)</code> provides level-based 3-color palettes while preserving default fallback | ✓ Verified              |
| Task 3                          | Completing a cube produces a brief visible flash without external assets/dependencies                    | ✓ Verified              |
| Task 4                          | <code>bonus_life_threshold()</code> returns 1000                                                         | ✓ Verified              |
| Existing bonus-life bookkeeping | <code>bonus_awarded</code> logic remains unchanged                                                       | ✓                       |
| Task separation                 | Four separate commits                                                                                    | ✓                       |
| Personal repository             | Changes pushed to your fork                                                                              | ✓                       |
| Working tree                    | Clean after final push                                                                                   | □ Confirm after command |
| Before video                    | 10-second recording captured before changes                                                              | ✓                       |
| After video                     | 10-second recording still required                                                                       | □                       |
| Chat history                    | Export required for submission                                                                           | □                       |
| Lab-4 folder                    | Final submission files need to be organized there                                                        | □                       |

## One final caution

After the Task 4 push, **don't make any more source-code changes** unless the final regression test exposes a real problem. The implementation phase should now be complete.

The remaining work is the **final full-game regression test, 10-second after-change recording, chat-history export, and Lab-4 submission packaging**. The official handout requires the before/after videos, updated code, and exported chat history under the `Lab-4` folder.

 Lab\_4\_VibeCoding\_Student\_handout

Task 4 has been committed and pushed successfully, with all four task commits present and the working tree clean. Proceed with the final regression verification of the completed game. Check the four implemented features together with the existing gameplay behavior, and identify any issue that must be resolved before the final after-change recording.

Yes. Since all four commits are already pushed and the working tree is clean, the remaining step is a **final regression run**, not another code change.

I cannot execute the Pygame application on your local Windows machine from here, so I can't truthfully certify runtime behavior myself. Use this checklist and report any failure before recording the final video.

## Final regression checklist

Run:

```
</> cmd
```

```
python game.py
```

### 1. Pyramid projection — Task 1

Immediately inspect the pyramid.

**Expected:**

- Pyramid is horizontally symmetric.
- Each row expands evenly around the center.
- No sideways drift/skew.
- Player can still move between adjacent cubes.

This confirms the `row / 2` correction didn't break movement.

---

## 2. Level-based palette — Task 2

Play until you complete the current level.

### Expected:

- Cubes progress through their painting stages normally.
- After the level is cleared, press `Space`.
- The next level appears.
- The palette changes between levels.
- Gameplay continues normally.

Also make sure the cubes don't disappear or become incorrectly colored.

---

## 3. Completion flash — Task 3

Paint a cube through its final stage.

### Expected:

</> Plain text

```
normal cube
↓
final painting
↓
brief visible white flash/outline
↓
normal completed cube
```

Verify that:

- the flash is brief;
  - it appears on the cube that was just completed;
  - it doesn't continuously flash;
  - completing another cube triggers another flash;
  - the game continues normally afterward.
- 

#### 4. Bonus life — Task 4

Verify the threshold behavior if you can reasonably reach the required score during normal play.

The implementation is:

</> Plain text

```
1000 points → +1 life
2000 points → +1 additional life
3000 points → +1 additional life
```

The existing `bonus_awarded` bookkeeping should prevent the same threshold from awarding lives repeatedly.

Also verify the original game mechanics that the README says must continue working:

The README specifically lists these as expected existing behaviors, so the four new features should not regress them.  
([raw.githubusercontent.com ↗](https://raw.githubusercontent.com/tytoir/tytoir/master/README.md))

## One thing I would check especially carefully

Task 3's flash implementation is the most likely place for a subtle regression, because it added state outside the original callback and rendering logic.

After testing a completion flash, make sure:

</> Plain text

flash appears

↓

flash expires

↓

game continues normally

and that the flash does **not** interfere with the normal cube drawing or player/enemy rendering.

---

## Before recording the final video

Also run:

</> cmd

git status

You should still get:

</> Plain text

nothing to commit, working tree clean

This is important because the final regression test should **not create any source changes**.

Then:

```
</> cmd
```

```
git log --oneline -4
```

You should have the four task commits:

```
</> Plain text
```

```
Add bonus life threshold
Add cube completion flash
Add level-based cube palettes
Fix pyramid projection
```

## Final decision

If all of the above pass:

**No further source changes are required.**

You are ready for the required **10-second after-change recording**. The recording should deliberately show the corrected pyramid and, where practical, the newly implemented behavior rather than simply showing a random 10 seconds of gameplay.

If **any** of the regression checks fail, don't record the final video yet—send me the exact behavior/error, and we'll determine whether it requires a Task-specific correction without disturbing the four-commit structure.

The final regression test has passed and the four implemented features are functioning correctly. Proceed with the final 10-second after-change recording. Specify the most effective gameplay sequence to capture the corrected pyramid and the implemented

features clearly, then confirm the files and Git state that should be preserved for the submission.

Great. Since the full regression test has passed, the code is now ready for the **after-change recording**.

The official handout requires a second **10-second video after all changes**.  Lab\_4\_VibeCoding\_Student\_handout

## Recommended 10-second recording sequence

The goal is to make the difference between the before and after recordings obvious while also showing the new functionality.

### 0–2 seconds — Show the corrected pyramid

Start with the game already running and the pyramid clearly visible.

Keep the full pyramid in view.

This immediately demonstrates **Task 1**:

The previously skewed pyramid is now symmetric.

### 2–4 seconds — Show player movement + painting

Make one or two normal hops across the cubes.

This demonstrates that the projection correction did **not break existing gameplay** and that cube painting still works.

### 4–6 seconds — Trigger a cube completion

If possible, complete a cube's final painting stage during this portion.

Try to make the brief **completion flash** visible.

This demonstrates **Task 3**.

## 6–8 seconds — Show the changed palette

If you are already close to completing the level, finish it and press `Space` so the next level appears.

The changed level palette demonstrates **Task 2**.

If reaching the next level naturally doesn't fit comfortably into 10 seconds, prioritize the completion flash rather than rushing the recording.

## 8–10 seconds — Leave the corrected game clearly visible

Keep the game window unobstructed and let the resulting pyramid/level remain visible.

Don't open terminals or VS Code during the recording.

---

## What the final video should communicate

Ideally the viewer sees:

`</>` Plain text

Corrected symmetric pyramid

↓

Normal player movement

↓

Cube painting

↓

Completion flash

↓

Different level palette

You **do not need to demonstrate the 1000-point bonus life in the 10-second video** unless you can reach it naturally. Trying to artificially manipulate the score would make the recording less representative of the actual game.

The README's Task 4 requirement is the `bonus_life_threshold()` behavior; your implementation and regression testing establish that functionality. ([raw.githubusercontent.com](https://raw.githubusercontent.com) ↗)

---

## Save the recording

Use a clear filename:

 Plain text

Lab4\_After.mp4

You should now have:

 Plain text

Lab4\_Before.mp4

Lab4\_After.mp4

The official handout requires both before and after videos.  [Lab\\_4\\_VibeCoding\\_Student\\_handout](#)

---

## Preserve the final Git state

After recording, do not edit the source code.

Run:

 cmd

```
git status
```

Expected:

```
</> Plain text
```

```
nothing to commit, working tree clean
```

Then:

```
</> cmd
```

```
git log --oneline -4
```

You should retain the four separate task commits:

```
</> Plain text
```

```
<hash> Add bonus life threshold
```

```
<hash> Add cube completion flash
```

```
<hash> Add level-based cube palettes
```

```
<hash> Fix pyramid projection
```

Also verify the remote:

```
</> cmd
```

```
git remote -v
```

It should point to **your personal fork**, not SETAPESU26/35\_qbert .

## Files to preserve for final submission

Your final submission package should contain:

</> Plain text

```
Lab-4/
|
├─ Lab4_Before.mp4
├─ Lab4_After.mp4
├─ game.py
└─ chat_history.pdf
```

The official handout specifically requires the Lab-4 folder to contain:

- **Videos — before and after**
- **Updated Code**
- **Chat history exported as a DOC/PDF**  Lab\_4\_VibeCoding\_Student\_handout

The repository README additionally asks for the complete LLM/chat history and a page link where applicable, so preserve that as well.

## Final state

</> Plain text

|                    |                                              |
|--------------------|----------------------------------------------|
| Source code        | <input checked="" type="checkbox"/> Complete |
| Task 1 commit      | <input checked="" type="checkbox"/>          |
| Task 2 commit      | <input checked="" type="checkbox"/>          |
| Task 3 commit      | <input checked="" type="checkbox"/>          |
| Task 4 commit      | <input checked="" type="checkbox"/>          |
| Regression testing | <input checked="" type="checkbox"/>          |
| Before video       | <input checked="" type="checkbox"/>          |
| After video        | <input type="checkbox"/> Record now          |

|                         |                       |
|-------------------------|-----------------------|
| Git working tree        | ✅ Should remain clean |
| Chat history export     | 📄                     |
| Lab-4 submission folder | 📁 Final packaging     |

**Record Lab4\_After.mp4 now, then leave the repository untouched.** The next step will be organizing the final submission materials and doing one final submission-readiness check.

The after-change recording has been captured and the final Git state remains clean. Proceed with the final Lab 4 submission packaging: organize the before/after videos, updated 35\_qbert source code, and exported chat history according to the official handout. Then provide a concise final checklist to verify that nothing required is missing before I push the Lab 4 folder to the submission repository.

Yes. At this point, the implementation and testing are complete. The remaining job is **submission packaging only**.

The official handout requires the Lab-4 folder in your Lab repository to contain:

1. Before video
2. After video
3. Updated code
4. Exported chat history as DOC/PDF  Lab\_4\_VibeCoding\_Student\_handout

## Recommended final structure

In your submission repository, create:

</> Plain text

```
Lab-4/
├─ before.mp4
├─ after.mp4
```

```
├─ game.py
└─ chat_history.pdf
```

I recommend these exact names because they are simple and unambiguous.

### 1. **before.mp4**

Use the **10-second recording captured before any source modifications**.

Do not replace it with a later recording.

### 2. **after.mp4**

Use your **10-second recording after all four tasks were completed**.

### 3. **game.py**

Use the final `game.py` from your completed `35_qbert` repository.

It should contain all four completed tasks:

- `cube_center()` — Task 1
- `cube_palette(level)` — Task 2
- `on_cube_completed(cell)` + minimal supporting flash rendering — Task 3
- `bonus_life_threshold()` — Task 4

You don't need to include `.git`, `.gitignore`, or the entire repository inside `Lab-4` unless your submission system specifically asks for the complete repository. The handout says **Updated Code**, so the final source file is sufficient based on the provided instructions.

 [Lab\\_4\\_VibeCoding\\_Student\\_handout](#)

### 4. **chat\_history.pdf**

Export the relevant ChatGPT/Vibe Coding conversation showing the prompts and implementation process.

Keep the complete Task 1 → Task 4 interaction rather than only the final code snippets. This is especially appropriate because the lab objective explicitly involves using Vibe Coding prompts and the handout requires the chat history export.

 Lab\_4\_VibeCoding\_Student\_handout

 Lab\_4\_VibeCoding\_Student\_handout

## Before pushing Lab-4

From your submission repository, verify:

 Plain text

Lab-4/

|— before.mp4 

|— after.mp4 

|— game.py 

|— chat\_history.pdf 

Then verify the actual Git state of the completed 35\_qbert repository one last time:

 cmd

```
git status
```

Expected:

 Plain text

```
nothing to commit, working tree clean
```

And:

 cmd

```
git log --oneline -4
```

Expected four task commits:

 Plain text

```
Add bonus life threshold
Add cube completion flash
Add level-based cube palettes
Fix pyramid projection
```

This satisfies the handout's explicit requirement for **separate commits for each task**.  [Lab\\_4\\_VibeCoding\\_Student\\_handout](#)

Also confirm that you pushed to **your personal repository**, not the SETAPESU26 main repository. The handout explicitly says not to raise a PR to the main repository.  [Lab\\_4\\_VibeCoding\\_Student\\_handout](#)

## Final submission checklist

| Requirement                      | Final status                                                                          |
|----------------------------------|---------------------------------------------------------------------------------------|
| Individual assignment            |  |
| Original game run before changes |  |
| 10-sec before video              |  |
| Task 1 — pyramid projection      |  |

| Requirement                    | Final status |
|--------------------------------|--------------|
| Task 2 — level-based palettes  | ✓            |
| Task 3 — completion effect     | ✓            |
| Task 4 — bonus-life threshold  | ✓            |
| Full regression test           | ✓            |
| 10-sec after video             | ✓            |
| Four separate task commits     | ✓            |
| Personal GitHub repository     | ✓            |
| Working tree clean             | ✓            |
| before.mp4 in Lab-4/           | Verify       |
| after.mp4 in Lab-4/            | Verify       |
| Final game.py in Lab-4/        | Verify       |
| Chat history PDF/DOC in Lab-4/ | Verify       |
| No PR to SETAPESU26            | ✓            |

## One final point

The handout is the authoritative source for the submission deliverables, and it specifically says **videos + updated code + chat history exported as DOC/PDF** under Lab-4 .  Lab\_4\_VibeCoding\_Student\_handout

So once those four files are physically inside Lab-4/ , and you verify the submission repository before pushing, **Lab 4 is submission-ready**.

The submission materials are now organized and verified. Provide the exact final Git commands to add the Lab-4 folder to the submission repository, create a single professional Lab 4 submission commit, push it to the personal repository, and perform the final verification.

Assuming you are currently inside the **submission repository** that should contain the Lab-4 folder, use these commands in order.

## 1. Confirm you're in the correct repository

 cmd

```
cd path\to\your\submission-repository
git remote -v
git status
```

Make sure `origin` points to **your personal submission repository**.

## 2. Check the Lab-4 contents

 cmd

```
dir Lab-4
```

You should see:

</> Plain text

```
before.mp4
after.mp4
game.py
chat_history.pdf
```

### 3. Add the complete Lab-4 folder

</> cmd

```
git add Lab-4
```

Before committing, verify exactly what will be submitted:

</> cmd

```
git status
```

You should see the four Lab-4 files staged.

For an additional check:

</> cmd

```
git diff --cached --stat
```

### 4. Create the single professional submission commit

</> cmd

```
git commit -m "Submit Lab 4 Vibe Coding deliverables"
```

This should be **one submission commit** containing the Lab-4 deliverables.

## 5. Push to your personal repository

If your submission branch is `main` :

</> cmd

```
git push origin main
```

## 6. Final verification

Check the latest commit:

</> cmd

```
git log -1 --oneline
```

Expected:

</> Plain text

```
<commit-hash> Submit Lab 4 Vibe Coding deliverables
```

Then:

</> cmd

```
git status
```

Expected:

</> Plain text

On branch main

Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

Finally, verify the remote:

</> cmd

```
git remote -v
```

Confirm it is your **personal repository**, not SETAPESU26/35\_qbert .

## Final submission state

Your GitHub repository should now contain:

</> Plain text

submission-repository/

└─ Lab-4/

├─ before.mp4

├─ after.mp4

├─ game.py

└─ chat\_history.pdf

And the Git history should preserve the earlier four implementation commits in the 35\_qbert development repository, while the submission repository has this single final packaging commit:

 Plain text

Submit Lab 4 Vibe Coding deliverables

The official handout requires the Lab-4 folder to contain the before/after videos, updated code, and exported chat history.

 Lab\_4\_VibeCoding\_Student\_handout

Do not make any further source-code changes after this push unless your submission repository verification reveals a missing file.

The Lab 4 submission has been committed and pushed successfully. Perform one final verification of the submission repository and confirm that all required deliverables are present, the working tree is clean, and the Lab 4 workflow is complete.

Based on the completion confirmations you provided throughout the workflow, **Lab 4 is complete and submission-ready.**

Final submission verification

| Requirement                    | Status                                                                                |
|--------------------------------|---------------------------------------------------------------------------------------|
| Personal submission repository |    |
| Lab-4/ folder created          |  |
| before.mp4                     |  |
| after.mp4                      |  |
| Final game.py                  |  |
| Exported chat_history.pdf      |  |

| Requirement                                     | Status        |
|-------------------------------------------------|---------------|
| Four separate development commits for Tasks 1–4 | ✓             |
| Final submission commit                         | ✓             |
| Submission pushed to personal repository        | ✓             |
| Working tree clean after push                   | ✓             |
| PR to SETAPESU26                                | ✗ Not created |
| Final regression test                           | ✓             |
| Lab 4 workflow                                  | ✓ Complete    |

The official handout requires the Lab-4 folder to contain the **before/after videos, updated code, and exported chat history**, and requires separate commits for each task while prohibiting a PR to the main SETAPESU26 repository.  [Lab\\_4\\_VibeCoding\\_Student\\_handout](#)

Final workflow

</> Plain text

```
Assigned 35_qbert repository
 ↓
Personal fork / clone
 ↓
Baseline verified
 ↓
10-sec BEFORE video
 ↓
Task 1 → separate commit
```

↓  
Task 2 → separate commit  
↓  
Task 3 → separate commit  
↓  
Task 4 → separate commit  
↓  
Full regression test  
↓  
10-sec AFTER video  
↓  
Chat history exported  
↓  
Lab-4 folder packaged  
↓  
Single submission commit  
↓  
Pushed to personal repository

**You can now submit Lab 4. No further Git or source-code changes are required.**